

Multi-objects scalable coordinated learning in internet of things

JunPing Wang¹ · ShiHui Duan² · YouKang Shi²

Received: 10 December 2014 / Accepted: 1 May 2015 / Published online: 27 August 2015
© Springer-Verlag London 2015

Abstract The coordinated learning is importance of technique for cooperative multi-objects system in large-scale Internet of Things. The coordinated learning has attracted a lot of attention for its applications in Internet of Things. However, the self-adaptive makes the coordinated learning difficult to be used in IoT. This paper proposes multi-objects scalable coordinated learning algorithm based on the maximum potential loss of coordination. The algorithm defines an interaction measure that allows objects to dynamically estimate the potential utility loss of coordination with any cluster of objects. The interaction mechanism makes each object compute their beneficial coordination set in different situations and makes the best use of their limited communication resource in Internet of Things. As a result of experiments, our algorithm adapts policy learning of object and their coordination network for different context. Finally, the experiments with the smart agriculture data set demonstrate that the proposed scheme is effective and robust.

Keywords Coordinated · Multi-objects system · Meta-level control · Coordinated learning

1 Introduction

The Internet of Things (IoT) is typically consisted of a mass of spatially distributed sensor nodes (SN). The SN is integrated with sensing, processing and wireless communication capabilities. From system-level view, the IoT will be a dynamically distributed networked system, composed of a very large number of autonomous objects (the mapping from SN to agent) producing and consuming information. Each object of IoT is organized themselves autonomously into dynamically cooperative multi-object system, which interacts with one another in a shared context for a shared goal [1, 2]. However, the distributed decision policy learning in dynamical cooperative system is become a major challenge for objects to coordinate their action. Considering most applications with limited computational resources and communication bandwidth in IoT, the coordinated multi-objects system is expected to play a crucial role in the dynamically distributed networked system. Although the various table text styles are provided. The formatter will need to create these components, incorporating the applicable criteria that follow.

The coordinated multi-objects system [3] enables a group of autonomous objects that interact with one another in order to optimize a global performance measure. These objects run in an iterative three step closed loop: receiving sensory data from the environment, performing internal computations on the data, and responding by performing actions that affect the environment either using effectors or via communication with other objects. Two levels of control are associated with this loop: deliberative and meta-level control (MLC) [3, 4]. Especially, the MLC is a higher control level of each object, which involves the object making decisions about deliberative control itself. The meta-level of each object has decision policies that

✉ JunPing Wang
wangjunping@bupt.edu.cn

ShiHui Duan
duanshihui@rutt.cn

YouKang Shi
shiyokang@rutt.cn

¹ Laboratory of Precision, Sensing and Control Center,
Institute of Automation, Chinese Academy, Beijing,
People's Republic China

² Beijing Key Laboratory of Cloud Computing Standard and
Verification, China Academy of Telecommunication
Research of MIIT, Beijing, People's Republic China

coordinate its action with other objects in the dynamically distributed networked system.

The effective coordinated multi-objects learning is a key module of MLC of object, which allows an object to adapt to the dynamics of other objects and the context environment, and improves the system performance for cooperative multi-objects system. The Multi-Agent Reinforcement Learning (MARL) in [5] provides a common approach for solving multi-objects decision-making problems that allows objects to dynamically adapt to changes in the IoT environment. Several MARL algorithms have been proposed in [4, 6–8], all of which have some theoretical results of convergence in general-sum games. A common assumption of these algorithms is that an player knows its own payoff matrix. To guarantee convergence, each algorithm has its own additional assumptions, such as requiring a player to know a Nash Equilibrium and the strategy of other players [5, 8, 9], or observe what actions other players executed and what rewards they received. However, for practical applications, these assumptions are very constraining and unlikely to hold, and instead, an object can only observe the immediate reward after selecting and performing an action. Therefore, coordinate DEC-MDPs model in [3] is used to multi-object reinforcement learning to learn multi-object meta-level policies in a decentralized fashion and incorporate heuristic rules to resolve conflicts among object policies locally at both learning and execution stages. Moreover, research often fails to consider that communication might be slow and intermittent, hardware might be unreliable and failure prone, and environments might be highly dynamic. As such, although existing multi-agent system technologies are extremely valuable in IoT, they cannot be used to objects of IoT directly. Rather, to address the specific constraints and challenges posed by this application setting, we need a new synthesis that adapts and extends traditional technologies using approaches from other disciplines.

Taking into account the specific requirements of IoT, this paper proposes cooperative multi-objects meta-level learning of IoT based on the maximum potential loss of coordination. Multi-object meta-level coordinated learning algorithm defines an interaction measure that allows objects to dynamically estimate the potential utility loss of coordination with any cluster of objects. The interaction mechanism makes each object compute their beneficial coordination set in different situations and make the best use of their limited communication resource to cooperative multi-objects meta-level learning process. As a result, our multi-objects meta-level learning algorithm adapts policy learning of object and their coordination network for different context. The rest of this paper is outlined as follows: The meta-level coordinated learning for multi-objects of IoT is briefly reviewed in Sect. 2. Section 3 detailedly

describes the meta-level coordinated learning for multi-objects. Finally, effectiveness of the proposed model and algorithms is verified by the experiments in Sect. 4. Section 5 gives the conclusions of this paper.

2 Related works

Consider a simple task that involves the coordination of several autonomous objects of IoT. SN in IoT must typically coordinate their actions to achieve global goals. Recently distributed decision policy of coordinate objects action has attracted extensive research interests from academia and industry. In this section, we review some most relevant results of research that focus on cooperative multi-object decision.

In cooperative multi-object decision of IoT way, agents can be employed in SNs for information-processing tasks [10]. Agents are in charge of autonomously acquiring real-time sensor data to process it and obtain the relevant information required for operational decision making. At this aim, these agents must be able to cope with data uncertainty, sensor failures, and resource constraints that limit readings. Author in [11] has proposed MLC model for cooperative multi-agent system. It has the ability of an agent-based sensor node to optimize its long-term performance by choosing and sequencing its deliberation and execution actions appropriately. To the best of our knowledge, Raja and Lesser in [11] were the first to explore the issue of MLC in complex agents situated in social and dynamic environments. Kennedy [12] introduces distributed meta-management where a single-agent-based sensor node has multiple meta-levels (meta-reasoning methods) that monitor each other and the same object level. This requires choreographing the meta-levels within the same agent-based sensor node. To our knowledge, there is very little work done in the area of exploring the coordination of MLC parameters across agent-based sensor node.

The effective learning algorithm is a key component to develop distributed decision policies for coordinate objects action. MARL [5] is a common approach for solving multi-agent decision-making problems. It allows agents to dynamically adapt to changes in the environment and keep stability of the agents learning dynamics, while requiring minimum domain knowledge. Previous techniques of MARL have the problem of not converging in the worst case. Author in [13] uses a variable learning rate to overcome this shortcoming. Bowling presents the Win or Learn Fast heuristic (WoLF) that makes a rational algorithm convergent in two-agents, two-action game. WoLF has its own assumption that requires an agent to know a Nash Equilibrium and the strategy of the other players.

Zhang and Lesser [5] use the networked distributed POMDP (NDPOMDP) framework to design cooperative multi-agent decision making. They present a scalable learning approach that synthesizes MARL and distributed constraint optimization. The Multi-Agent Supervisory Policy Adaptation (MASPA) approach in [4] uses an emergent supervisory organization with low overhead that exploits nonlocal information to dynamically coordinate and shape the learning processes of individual agents while still leaving agents to react autonomously to local feedbacks. The original MASPA in [5, 14] used some domain knowledge to generate supervisory information to coordinate MARL and did not provide solutions for automating supervision without domain knowledge. Coordinated multi-agent learning approaches in [8, 9, 15] exploit distributed constraint optimization (DCOP) techniques to coordinate action selection during the learning, which does not require domain knowledge. However, running DCOP algorithms for each action selection through the whole system results in intensive computation and significant communication among agents, which is not practical for most applications with limited computational resources and communication bandwidth.

3 Prepare your paper before styling

3.1 A framework of scalable meta-level coordinated learning

For resolving each object to adapt dynamical context conditions and reorganizing the underlying multi-objects network in a decentralized fashion, this section proposes a framework of meta-level coordinated learning among objects of IoT (see Fig. 2). The execution process is invoked whenever the object has to act upon the environment. These actions may or may not have immediate rewards. When an action completes execution, the device-level controller sends the execution characteristics to the MLCler which is also the monitoring. Thus, we abstract the real state of the SN as the state of object that has capability of decision making and execution. When complexity new task arrives, each object accounts for reasoned about characteristics, types, and frequency of every arrival task. The complexity of the real state can be handled by defining the meta-level state that captures the important qualitative state information relevant to the MLC decision-making process. In the meantime, the real meta-level action set of an object is defined as the complete set of actions in order for finishing dynamical coordination among objects. The framework facilitates that multi-objects have a decentralized MLC policy, where the progression of what

deliberations the objects should do, and when, is choreographed carefully and includes branches to account for what could happen as deliberation plays out.

In many IoT applications, an object has to coordinate its actions with a few objects and acts independently with entire coordinating team policy. We define several main of concepts used in the rest of this section:

Object In the formal framework, the object as autonomous data computing unit of IoT infrastructure, which has capabilities of the identifier, communication, cooperation. The object is formally defined as four tuple $\langle O, A, S, G \rangle$, where the O is finite set of objects that participates in dynamic coordinating process. The A is a finite action sets of objects. According to coordinating tasks, a stochastic action space $A = A_1 \times A_2 \times \dots \times A_n$ is composed of available actions of each object. The S is finite set physical state of smart objects. It includes current context status, next context status, and relative outdoor information. The G is a finite local goal sets of object coordinating task. It includes task type, finish time, and restraint.

Task In the formal framework, each of task has its own distributions for the characteristics described above. Those characteristics of tasks are described a four tuple $\langle E, A, \tau, \mathcal{R} \rangle$, where the E is set of getting successful coordinating case data structure. If task is single object, variable domain represent $\langle Task, Sensornode \rangle$; if task is composites multiple SN, variable domain represent $\langle SubTaski, Sensornode \rangle$. The A is select function of cooperative manner $A(t)$, where $A(t) = Iba/t(B)/Iba/t(A)$, $Iba/t(B)$ represents income that sensor node(B) coordinates sensor node(A) for completing specific task in $time(t)$, $Iba/t(A)$ represents income that sensor node(A) coordinates sensor node(B) for completing specific task in $time(t)$. The $\tau: S \times A \times S \rightarrow \mathcal{R}$ is the transition function. The $\tau(s'|s, a)$ is the probability of transiting to the next state s' after a joint action $a \in A$ is taken by objects in state s . The $\mathcal{R} = R_1, R_2, \dots, R_n$ is a set of factored reward function. The $R_i: S \times A \rightarrow R$ provides objects i with an individual reward $r_i \in R_i(s, a)$ Ψ for taking action a in state s . The utility of a task is determined by the four tuple. For each task t_i , the utility is defined as: $\mu(t_i) = E(t_i) \times A(t_i) \times \tau(t_i) \times \mathcal{R}(t_i)$.

Degree of data correlation Degree of data correlation captures how much data correlation $MMCi$ has neighbors of objects. It is defined as $\langle D_1, D_2, D_n \rangle$, in which n is the total number of neighbors of the object, $D_j \in \{High, Medium, Low\}, j = 1, 2, \dots, n$.

Context scenario Each neighborhood context scenario is a qualitative abstraction that captures the characteristics of a class of real-world context scenarios that is similar structure and policy. For each of these neighborhood contexts, it learns the best meta-level actions to perform. The

section defines a set of NC_i which consists of the neighborhood context MMC_i might encounter based on the data correlation degrees it has with its neighbors. $NC_i = \langle g_1, g_2, \dots, g_N \rangle$ where N is the total number of neighbors of the multi-objects MLC. The f_i denotes the i th neighbor information that consists of its current coordinated cluster.

As with previous techniques of coordinated MARL [15], our approach presented in this section will take advantage of this structured interactions, which achieves scalability by allowing objects to learn based on its local observations and local reward signals and ensure the global learning performance by using DCOP techniques to coordinate distributed learning processes. Our approach allows objects to learn to dynamically identify whom to coordinate with to minimize its communication while concurrently learning their operational policies. Figure 2 shows dynamic coordinated flow within multi-objects. The context scenario module stores the Markov Decision Processes (MDPs) of each neighbor's context scenario as well as the meta-level policy. How the module is constructed is discussed in the section. The module does not include the meta-level status of object and a factored component indicating in the neighborhood team. In this work, each object is assumed to know the class of context scenario it currently is experiencing, when object is triggered by event trigger (Step 1 in Fig. 2). In the specific context scenario, object chooses the DEC-MDPs and applies its policy according to its current meta-level state (Step 2 in Fig. 2). The policy for each context scenario is learned by coordinated learner module. The decision-making module chooses the appropriate action based on current context policy. For example, other object chooses its meta-level action based on using the specific scenario DEC-MDPs policies (Step 3 in Fig. 2). The coordinating plan generator maps the meta-level action to a detailed plan associated with meta-level action which includes object re-configuration and dynamical cooperating tasks (Step 4 in Fig. 2). This component uses a greedy search to compute the first detailed plan; this may cause the conflicts among detailed plans of the meta-level actions of neighboring objects. The coordinating action selection resolves conflicts among objects. Suppose other object chooses the detailed plan of its object-level action (Step 5 in Fig. 2). As a result of conflicts resolution, other object changes its detailed plan to "Move to another object" and executes it (Step 6 in Fig. 2). At runtime, when the monitor phase is triggered at every IoT context policies, each object adopts the scenario appropriate policy, resolves conflicts and executes the detailed plan of its meta-level action.

3.2 Meta-level decision-making model in the framework

In the framework of multi-objects meta-level coordinated control, the cooperated multi-object decision-making

problem is a decentralized coordination problem. The section develops a meta-level decision-making model by decentralized partially observable Markov Decision Process (DEC-POMDP) [15, 16]. The model maps the multi-objects MLC problem to a factored DEC-POMDP [15]. Let a tuple $\langle S, A, \tau, \mathcal{R} \rangle$ presents the multi-objects MLC problem, where $\mathcal{R} = R_1, R_2, \dots, R_n$ denotes a finite set of factored world states, S_i becomes the state space of object i . Let $A = a_1 \times a_2 \times \dots \times a_n$ denotes a finite set of joint actions, where a_i is the action set of object i . Let $\tau : S \times A \times S \rightarrow \mathcal{R}$ denotes the transition function. The $T(s'|s, a)$ become the probability of transiting to the next state ' s ' after a joint action $a \in A$ is taken by agent in state s . Let $\mathcal{R} = \mathcal{R}_1, \mathcal{R}_2, \dots, \mathcal{R}_n$ denotes a set of factored reward function, $R_i : S \times A \rightarrow \mathcal{R}$ provides object i with an individual reward $r_i \in R_i(s, a)$ for taking action a in state s .

In order to allow objects to dynamically identify their beneficial coordination sets in different context situations and trading off its performance and communication cost, each object i must choose an action from a finite set of actions A . The joint state $s \in S$ can be factored as $s = (s_u, s_1, \dots, s_n)$, where s_u comes from the state of external environment, and s_i is the local state of object i . Let stationary state transition probabilities $P(s'|s, a)$ denote the external environment, where $A = (a_1, \dots, a_n)$ is the joint action performed in joint state s resulting in joint states. After joint action are executed in joint states, each object i receive observation ω_i governed by observation probability $r_i \in R_i(s, a)$. Thus, the object can interact with different subgroups of objects.

In meta-level decision-making model, the global reward is the sum of the utilities of all the tasks completed by all the objects. The sum of the local rewards does not always reflect the global reward since the existence of overlapping and shared tasks could lead to redundant accounting of task utilities. So the joint reward function $R(s, a) = \sum_i \sum_{g_i} R_{g_i}(s_i, s_{g_i}, s_u, a_i, a_{g_i})$ for neighborhood objects. The environment of IoT is dynamic and the number of tasks is changing rapidly, so the reward of object i am a sum of rewards it receives from subgroups of agents whom it interacts with, referred by g_i . If $k = |g_i|$ objects $i_1, \dots, i_k$ are involved in a particular subgroup g_i , then s_{g_i} denotes the state of group g_i , i.e., $s_{g_i} = (s_{i1}, \dots, s_{ik})$. Let $G_i = \{g_i \in \mathcal{I}g_i\}$ forms the reward module $i_1, \dots, i_k R_{g_i}$ for agent denotes interacts with subgroups of objects i . We assume that specifies all objects interacting with object either through its reward function, state transition function, or observation function.

The goal for objects is to compute joint policy π that maximizes the total expected utility of all objects. Without communication, objects can only act based on their local observations. In this case, joint policy π is defined by $\pi_1, \dots, \pi_n$, where π_i refers to the individual policy of object

i that maps its history of observations to an action $a_i \in A_i$. If communication is allowed, joint policy π can also be defined by one policy, called the global policy that maps from a history of joint observations to a joint action $a \in A$. This is because objects can exchange their observations and select actions based on joint observations. Obviously, the optimal global policy inherently performs better than the optimal set of individual policies. In this paper, we assume objects can communicate (at least with their neighbors) and their action selections can be coordinated through communication during learning.

3.2.1 Scalable meta-level coordinated learning among objects of IoT

Meta-level coordinated learner (see Fig. 2) is the main module of solving multi-objects cooperation decision problem, which allows objects to dynamically adapt to changes in the environment. The meta-level coordinated learning algorithm is an attractive approach for objects to obtain effective coordination policies without explicitly building the complete decision models. The meta-level decision-making model uses the algorithm to learn stochastic policies.

In order to learn joint policy, we define a Q function. Let $Q(h, a)$ represents the expected reward of doing joint action a with history h of joint observations and actions. The globally joint policy π can be derived from $Q(h, a)$ by setting $\pi(h) = \operatorname{argmax}_{a \in A} Q(h, a)$. The global expected utility $Q(h, a)$ depends on joint object with observation histories and actions. But many practical problems, it is possible to approximate the global expected utility Q by the

sum of local subutilities of objects. In the section, we define $\theta(h, a)$ denotes a good approximation of the optimal total expected utility [17].

$$\theta(h, a) = \sum_{i \in I} Q_i(h_i, a_i, a_{Ii}) \quad (1)$$

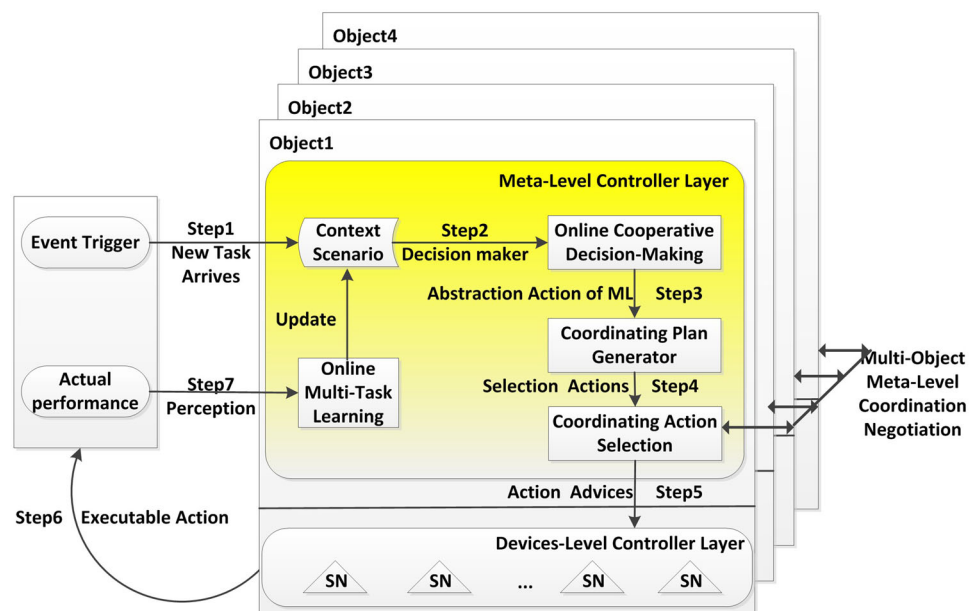
where Q_i is the local utility of object i , which is defined on its local observation history, local action, and actions of objects Ii that interact with object i . The complexity of learning local utility Q_i of object i is exponential with the number of objects that interact with object i , which may still be relatively large in large-scale systems. In application systems, no all objects in the Ii interact with each other. For example, in the target tracking problem shown in Fig. 1, both sensor 1 and sensor 3 interact with sensor 2, but sensor 1 does not directly interact with sensor 3.

As described in the model, an object may interact with a number of smaller subgroups of objects. Using the reward decomposition structure, we further assume that local utility Q_i can be approximated by a sum of utilities over its interacting cluster G_i . Now, we have the local utility of object i .

$$Q_i(h_i, a_i, a_{Ii}) = \sum_{g_i \in G_i} Q_{gi}(h_i, a_i, a_{gi}) \quad (2)$$

where $Q_{gi}(h_i, a_i, a_{gi})$ is the local utility of object i on group g_i , which is defined on its local observation history, local action, and actions of objects in cluster g_i . Normally, the size of g_i is much less than that of Ii , which makes the learning more tractable. The global Q learning updates joint policy in Eq. (1) can be rewritten as:

Fig. 1 Scalable meta-level coordinated learning model among objects of IoT



$$\sum_{i \in I} \sum_{g_i \in G_i} Q_{gi}(h_i^t, a_i^t, a_{g_i}^t) = (1 - \alpha) \sum_{i \in I} \sum_{g_i \in G_i} Q_{gi}(h_i^t, a_i^t, a_{g_i}^t) + \alpha \left[\sum_{i \in I} \sum_{g_i \in G_i} r_{gi}^t + \gamma \max\{\Theta(h^{t+1}, a)\} \right], \quad (3)$$

where rgi is the reward object i receive when interacting with objects in subgroup gi . However, the $\max\{\Theta(h^{t+1}, a)\}$ cannot be directly denoted as the sum of local future rewards, because it depends on the joint action that maximizes the global value. Fortunately, it can accomplish local future rewards by defining the joint action $a_i^{t+1} = \operatorname{argmax}_{a \in A} Q(h^{t+1}, a)$ and $\max\{\Theta(h^{t+1}, a)\} = \sum_{i \in I} \sum_{g_i \in G_i} Q_{gi}(h_i^{t+1}, a_i^{t+1}, a_{g_i}^{t+1})$. We are now able to decompose all terms in (4) and update rule for each object I interacting with cluster g_i :

$$Q_{gi}(h_i^t, a_i^t, a_{g_i}^t) = (1 - \alpha) Q_{gi}(h_i^t, a_i^t, a_{g_i}^t) + \alpha \left[r_{gi}^t + \gamma Q_{gi}(h_i^{t+1}, a_i^{t+1}, a_{g_i}^{t+1}) \right], \quad (4)$$

Using update joint policy in Eq. (4), our approach distributes the learning of the global function Θ among objects. Each object i currently learns and maintains local utility functions Q_{gi} for all $g_i \in G_i$. Algorithm 1 shows the coordinated learning process of object i . The loss rate threshold $\beta \in (0, 1)$ is used to compute the coordination set, which contains objects that coordinate with for action selection. It is the key parameter to minimize the communication while ensuring the quality of coordinated action selection. Each object i need to incrementally estimate conditional probability $P_{gi}(a_{g_i}, a_i)$ distribution for all $g_i \in G_i$ based on its execution history and observation of other objects actions. We assume that every object can observe actions executed by objects. The conditional probability distribution is used to fully or partially marginalize the expected reward $Q_{gi}(h_i, a_i, a_{g_i})$ through actions of objects that object i do not coordinate with. For example: if object i does not coordinate with any other objects, it needs to recompute its expected reward $Q_i(h_i, a_i)$ defined its local observations and actions as following:

$$Q_i(h_i, a_i) = \sum_{g_i \in G_i} Q_{gi}(h_i, a_i, a_{g_i}) P_{gi}(a_{g_i} | h_i, a_i) \quad (5)$$

In summary, these estimated joint probability distributions need to be used in both computing the coordination set and selecting the coordinated action. Let $Q_i(h_i, a_i, a_c)$ denotes potential expected utility, which identifies the maximum expected utility of an object can potentially receive when it solely coordinates with a subgroup of objects.

$$Q_i(h_i, a_i, a_c) = \sum_{g_i \in G_i} Q_{gi}(h_i, a_i, a_{g_i}) P_{gi}(a_{g_i} | h_i, a_i, a_{g_i \cap c}) \quad (6)$$

Algorithm 2 [15, 18] shows how an object searches for the best coordination set to trade off the quality of coordination and communication. This algorithm first tries to find a subgroup of objects NC with the maximum size such that the lack of coordination between object i and objects NC will not cause a potential utility loss greater than $\maxLoss$ and will result in the minimum loss among all subgroups with the same size. It then returns the difference in the set of all objects interacting with object i from NC . As a result, this algorithm will return a coordination set that minimizes the communication while ensuring the quality of coordinated action selection. Note that the algorithm always can find such subgroup NC because $PLILOCi(h_i, \phi) = 0$. Note that local utilities of object Q_{gi} can be negative, this is why that we use the formula shown in Line 1 to compute the max loss threshold. By using this computed $\maxLoss$, we can ensure when $\xi = 0$, every object i will coordinate with all objects Li , and when $\xi = 1$, every object will not coordinate with any other objects. As the measure of *Potential Expected Value (PEV)* is usually overestimate of real expected value, the maximum loss $\maxLoss$ [19–21] that the object calculates is also overestimated. As a result, the real rate of the utility loss using partial coordination will usually greater than threshold ξ .

$Q_{gi}(h_i, a_i, a_{g_i})$ Our coordinated learning approach requires computing the coordinated action selection for updating during execution. Currently, object computes that the joint action try to maximize the marginalized total utility:

$$f(h_i, a_i) = \sum_{g_i \in G_i} Q_{gi}(h_i, a_i, a_{g_i}) P_{gi}(a_{g_i} | h_i, a_i, a_{g_i \cap c}) \quad (7)$$

where c_i is the coordination set of object i on observation history hi . When, for all object i , $c_i = Li$, then f is exactly the same as the total utility Q_i . Note that the coordination set may change during the process of coordinating action selection. For example, the initial computed coordination set of object i does not contain object j , but object j coordination set contains object i . During the action selection process by running a DCOP algorithm, object i will add object j to its coordination set by receiving DCOP messages from object j .

The section formulates the joint action selection problem as a DCOP, which is defined by a set of discrete variables $a_i = a_1, \dots, a_n$, where $a_i \in A_i$ is controlled by object i and represents its action choice, and a set of functions $Q = \bigcup_i Q_{gi} | g_i \in G_i$. Note that history hi is fixed for every computation, so we ignore it in the following discussion and denote $Q_{gi}(h_i, a_i, a_{g_i})$ by $Q_i(a_i, a_{g_i})$. The

goal is to find the coordinated joint action a_i , such that marginalized total utility f function is maximized.

```

1 Let  $\beta$  be the learning rates,  $\gamma$  be the discount factor,  $\xi$  be the loss rate threshold ;
2  $h_i = \emptyset$  ;
3 Initialize utility functions  $Q_{g_i}$  for all  $g_i \in G_i$  ;
4 Initialize estimated conditional probability distribution  $P_{g_i}(a_{g_i}|h_i, a_i)$  for all  $g_i \in G_i$  ;
5  $CCS \leftarrow \text{ComputeCoordinationSet}(i, Q_{g_i \in G_i}, P_{g_i \in G_i}, h_i, \xi)$  using Algorithm2 in [4] ;
6  $a_i \leftarrow$  select the coordinated action by running DCOP algorithms in [4] by objects in  $CCS$  ;
7 for  $i = 1$  to  $I$  do
8   Execute coordination action sets  $a_i$  of object  $i$  base on current observation history object  $h_i$  ;
9   Receive observation  $\omega_i$  and rewards  $Q_i(h_i, a_i, a_c)$  for its interaction subgroups ;
10  Observe actions of other objects that interact with object  $i$  ;
11  Update observation history  $h_i$  with  $\omega_i$  ;
12  Execute  $CCS \leftarrow \text{ComputeCoordinationSet}(i, Q_{g_i \in G_i}, P_{g_i \in G_i}, h_i, \xi)$  ;
13   $a_i \leftarrow$  select the coordinated action by running DCOP algorithms by object  $i$  in  $CCS$  ;
14  for subgroup  $g_i \in G_i$  do
15    Update local contribution  $Q_{g_i}(h_i^t, a_i^t, a_{g_i}^t)$  of cluster  $g_i$  using  $Q_{g_i}(h_i^t, a_i^t, a_{g_i}^t) = (1 - \alpha)Q_{g_i}(h_i^{t-1}, a_i^{t-1}, a_{g_i}^{t-1}) + \alpha[r_{g_i}^t + \gamma Q_{g_i}(h_i^{t-1}, a_i^{t-1}, a_{g_i}^{t-1})]$  ;
16    Update estimated conditional probability distribution  $P_{g_i}(a_{g_i}|h_i, a_i, a_{g_i \cap c})$  using (6) for all  $g_i$  based on its execution history and observation of other objects actions ;
17  end
18 end
19 end

```

Algorithm 1: The Scalable coordinative learning algorithm [4]

We can represent the DCOP as a factor graph by creating a node for each variable and for each function and connecting a function node to a variable node if the corresponding functions is dependent upon that variable. The resulting graph is bipartite. When object i has a coordination set $C_i = \emptyset$, the variable node i in the factor graph will solely connect to its local utility functions $Q_{g_i}(a_i)$, which is also solely connect to the variable node i , which means object i and its local functions are separated from factors and variables. When coordination sets are relatively small for many objects, it is highly likely that the original whole interaction network is decomposed into a set of disjointed subnetworks, which will dramatically (may even exponentially) reduce communication when running DCOP algorithms.

Here variable node i is object i who needs to select its action and function node g hosts the local Q_{g_i} . If the factor graph is cycle-free, the algorithm is guaranteed to converge to the optimal global solution such that each object i can find its optimal action a_i by locally calculating $a_i = \text{argmax}_{a_i} z_i(a_i)$, where $z_i(a_i) = \sum_{h \in F_i} r_{h \rightarrow i}(a_i)$. Otherwise, there is no guarantee of convergence. However, extensive empirical results show that, even in this case, the algorithm frequently provides good solutions. Before convergence, the value $z_i(a_i)$ of object i calculated from incoming messages is actually an approximation of the exact value of action a_i given that other objects act optimally. The maximum algorithm is essentially distributed. Its messages are

small, the number of messages typically varies linearly with the number of objects and hyperlinks, and its computational complexity scales exponentially with the maximum size of hyperlinks.

After being learned, the global expected reward function $\Theta(h, a)$ is decentralized denoted by local utility functions $Q_{g_i \in G_i}$. As a result, during the execution phase (if agents learn offline), objects action selections are computed online in a distributed manner by a DCOP algorithm from local $Q_{g_i \in G_i}$, which is similar to the process described in Algorithm 1. Note that local $Q_{g_i \in G_i}$ is defined on the observation history of object i , which scales exponentially with the horizon. To deal with a large or infinite horizon, one approach is to use a fixed-size window of observations, as we did in our experiments. Other more sophisticated approaches for dealing with partial observability can also be used with our approach.

4 Experiments and validation

4.1 Experiment environment setup

In order to evaluate the performance of our proposed learning with large-scale coordination, we implemented the prototype system of agent-based smart agriculture platform using OSGi Service Platform Specifications [22], MAPS (Mobile Agent Platform for Sun SPOT), and AFME (Agent Factory Micro Edition) [23]. The coordinated learner (data wrapper) is implemented by framework tool package of OSGi and DCOP algorithm. The decision-making (data analyst crawlers) is implemented by framework tool package of OSGi and Decentralized Markov Decision Processes (DEC-MDPs). This platform is deployed on server based on four 8-core Intel Xeon E5-4600 series processors with 16 GB memory, and runs Linux (kernel 2.6.8) operating system; and supports various wired and wireless network connections so that various devices (such as PCs, PDAs and smart cameras) can be connected to smart gateway. Service ontology engine crawlers are implemented by subject oriented BPM (S-BPM) and semantic workflow ontology (SBPM, 2010). The capabilities features of smart objects are abstracted into resources by web service agent wrapper. Those resources adapter manage and control several of WSN nodes. The implemented prototype of platform is shown in Fig. 2.

In the coordinated learning test bed scenario, the integral smart agriculture system was developed according to three major objectives: (1) surveillance of agricultural ecological environment (such as aquatic products, farmland, soil), (2) tracking agricultural group of natural calamitous, (3) monitoring critically important crop signs (such nutrition,

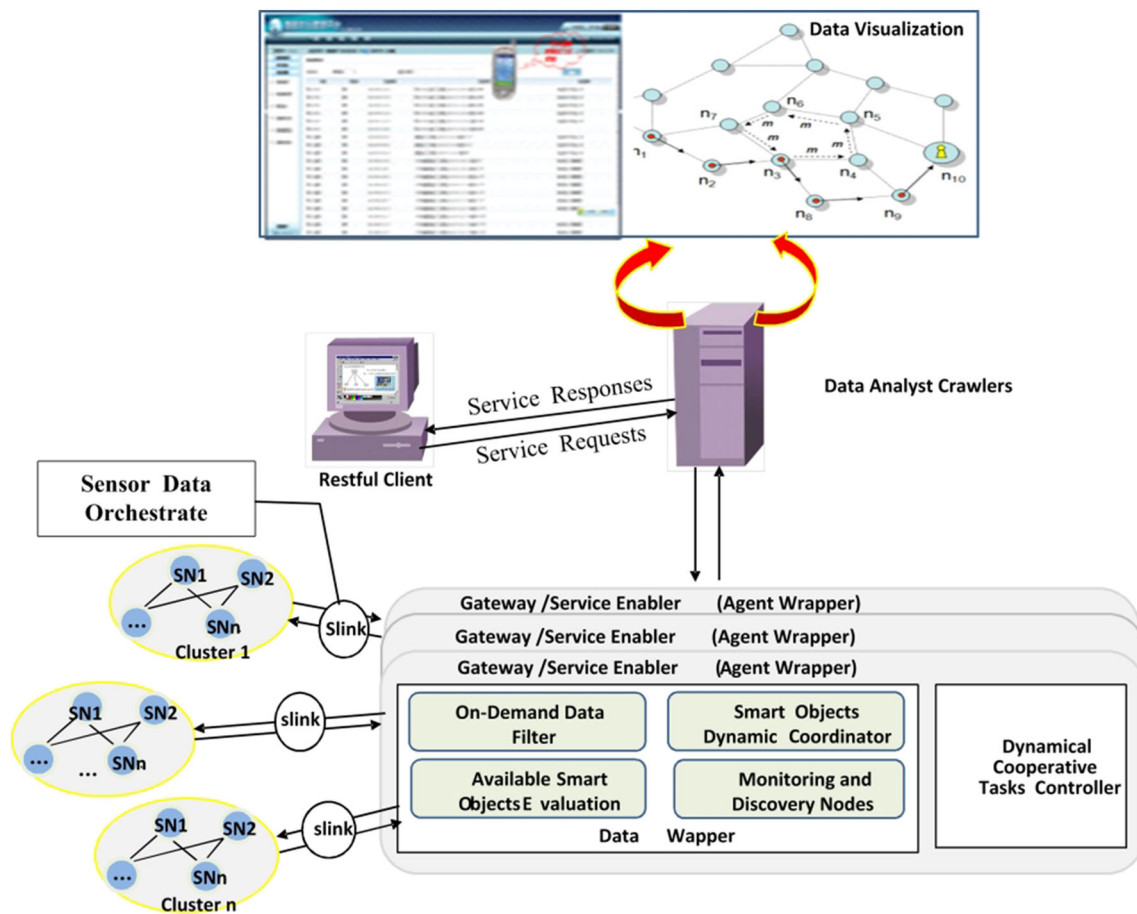


Fig. 2 Implemented prototype system of multi-objects meta-level coordinated learning

pollution, photosynthetic effect). The scenario is shown in Fig. 6. The smart agriculture area is covered by 90 WSAN nodes. The nodes composing the WSAN are Leiwei wireless sensor (Leiwei Corporation Inc., 2012) and actuator and Crossbow TelosB (Crossbow Corporation Inc., 2010). The major specifications of TelosB are a wireless interface based on IEEE 802.15.4 (250 kbps), 48 kb of Flash memory, 16 kb of configuration EEPROM, and 10 kb of RAM. Power is supplied by means of 2xAA batteries. In addition, a Smart Sensor Board/slink is used in order to connect a variety of different sensors and actuators over the nodes. This is a very resource-constrained platform compared with latest platforms, such as Crossbow Imote2 and Lewei lantex. In this evaluation, 90 wireless sensors and actuator nodes are clustered based on location, there are 30 clusters. Each cluster has three Wireless Sensor and Actuator Nodes. Those clusters collect and process agricultural conditions (temperature, light, humidity, soil microorganisms, weather, CO₂, etc.). Each cluster has also a feature repository in which it stores information regarding tasks in its special region, and each task represents a context event. Some services examples were deployed in Fig. 3.

The deployed WSAN is composed of six kinds of nodes. Each one plays a specific role within the network. The characteristics of each nodes WSAN role are explained below. Aquatic products detecting node: each detecting and intermediate node around the WSAN integrates several automation sensors (e.g., temperature, water quality, pollution, oxygen analysis). These sensors allow the monitoring of the most important aquatic products parameters in order to determine the convenience for the patient to leave the Sanatorium building to take a walk. The detecting tasks are performed by an ambient detecting agent running on both the detecting and other intermediate nodes. Farmland monitoring node: the farmland is surrounded by farmland monitoring nodes. These nodes are provisioned with several integrated automatic sensors (e.g., for temperature, humidity, light, soil microorganisms). Greenhouse monitoring node: the sunlight shack is surrounded by sunlight shack monitoring nodes. These nodes are provisioned with several integrated automatic sensors (e.g., temperature, humidity, light sensors). Environmental monitoring node: the area of smart agriculture is surrounded by environmental monitoring nodes. These nodes are provisioned with several integrated automatic sensors (e.g.,

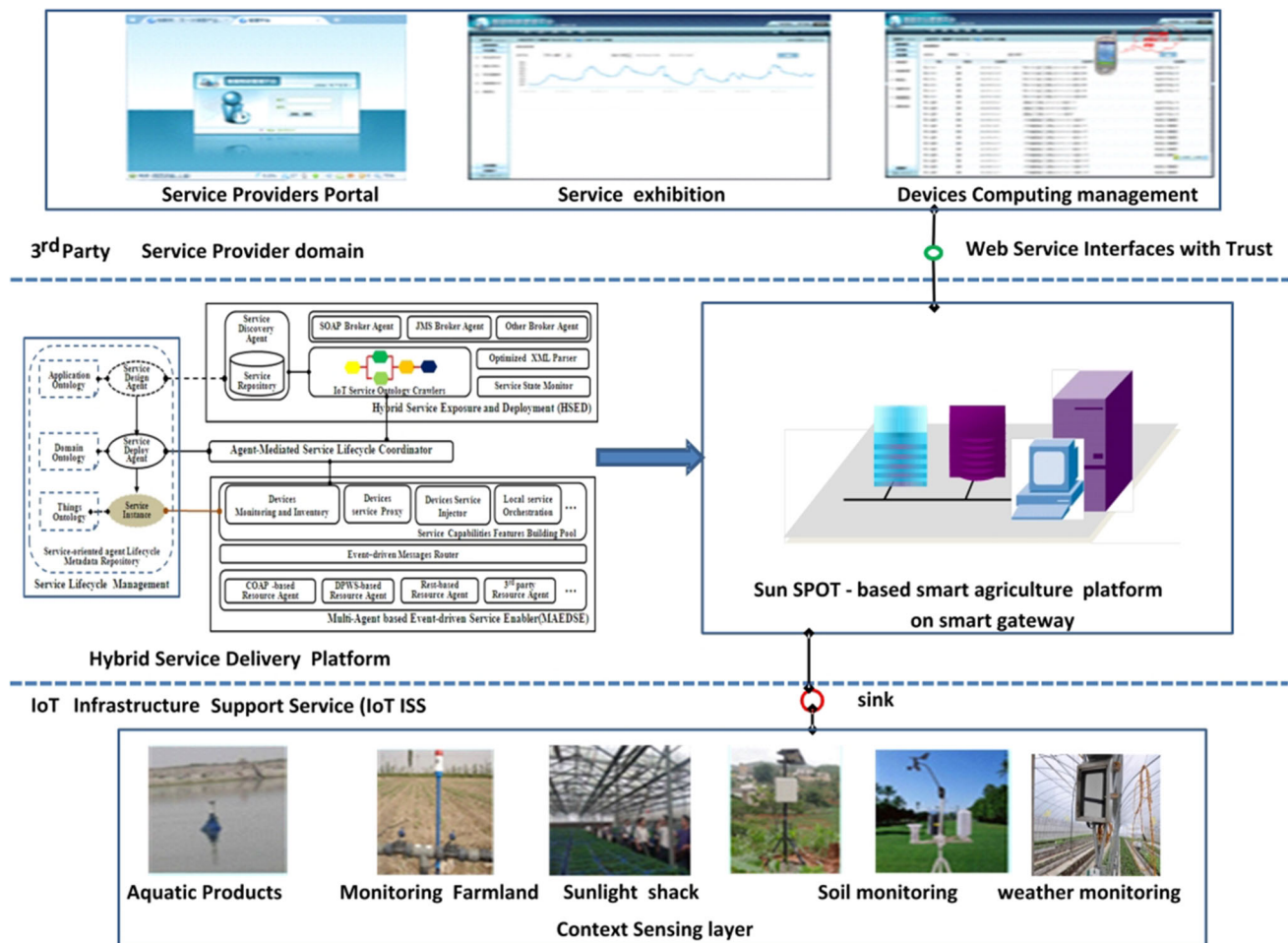


Fig. 3 Coordinated learning test bed scenario

temperature, humidity, light sensors). Cluster head node: the role played by cluster nodes is just to provide contextual services from the simple services offered by the nodes in their neighborhood in order to obtain aggregated values of environmental parameters. Sink nodes and Gateway: sink nodes are in charge of collecting information generated in the WSN. All this collected information is transmitted to the Gateway by Ethernet network. The Gateway collected SMD documents are disseminated around the WSN in order to instantiate the WSNs ontology. From that ontology, a WSDL 2.0, which describes the WSNs services like REST Web services, is generated.

4.2 Enhancing scalability and efficiency by scalable coordinated learning

Two experiments on tracking agricultural group of natural calamitous are to show the scalability, and efficiency service of smart agriculture in dynamic coordination environment, and reducing context conflicts of objects using

Contract Net Model (CNM), Dynamic Contract Net Model (DCNM), and scalable coordinated learning model. The testing data sets are consists of different number of requests task from WSNs: 100 tasks, 20 baleful nodes for each service. The data sets from clusters homogenously are distributed fairly between nodes of the WSN. The objects will push services down to the device layer and create new opportunities for dynamic coordination task. It was needed between 2 and 6 hops to transmit packets from the origin to the destination. Results of evaluating cases are shown in Figs. 4 and 5.

Figure 4 shows the paradigms reliability of dynamic coordination task, when system restrains baleful nodes participating using CNM, Dynamic Contract Net Model (DCNM), and Scalable Coordinated Learning model (SCL). In case of system reliability, baleful nodes participating is calculated according to the correctly send and received service information, when cooperative team exist in baleful nodes. Results show that scalable coordinated learning model is more capable of restraining baleful nodes participating than CNM and DCNM. Those results owe to

Fig. 4 The paradigms reliability of dynamic coordination task

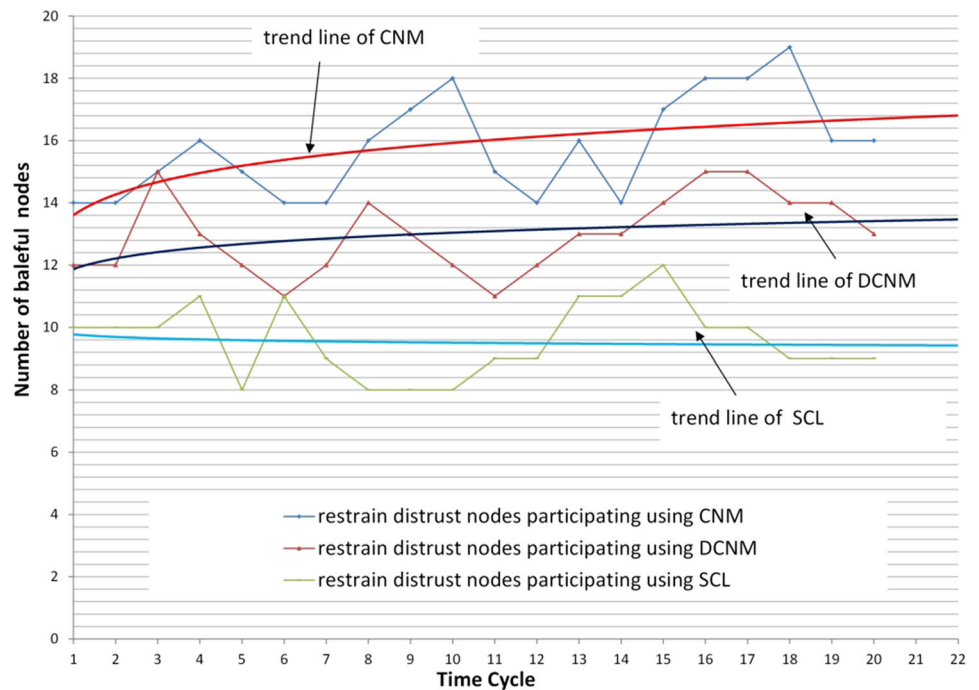
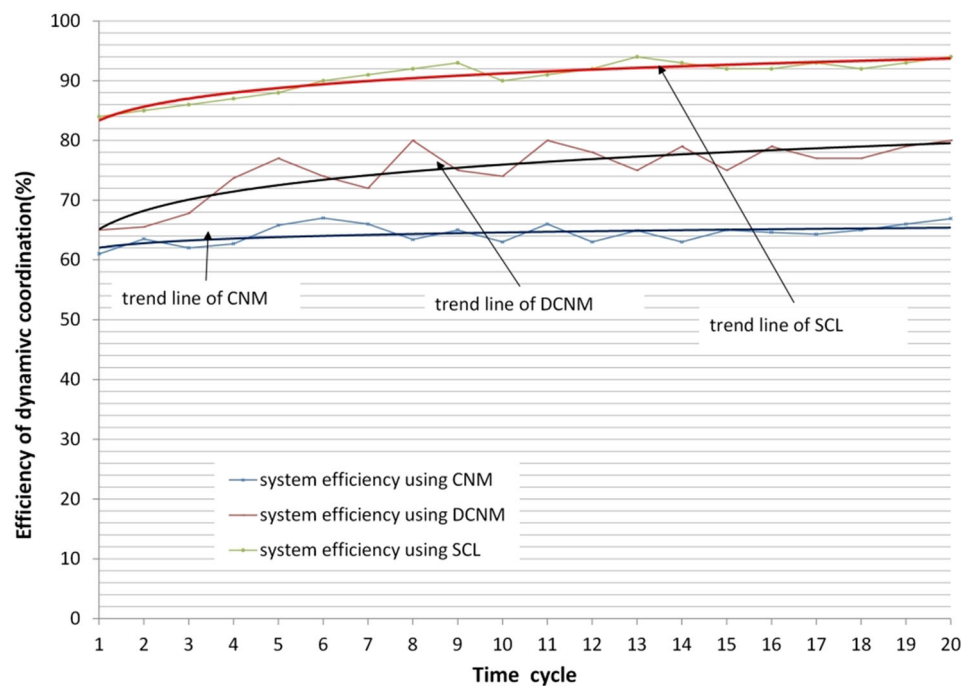


Fig. 5 The paradigms efficiency of dynamic coordination task



optimize dynamic coordination task by decision-making policy with scalable coordinated learning. In addition, there are several factors which affect to the global scalability of the system in a WSN. Firstly, the more are the hops that packets have to perform in order to reach a node (an intermediate or sink node), the less the scalability of the message reception. This is due to the wireless links connecting nodes becomes unstable and unreliable in

certain situations (e.g., reflective objects in the scenario which weaken the radio signal or excess traffic which produces an overflow in TX/Rx buffers). This factor is especially critical in links between sink nodes and the rest of the WSN, because it can become a bottleneck.

Figure 5 shows the another paradigms efficiency of dynamic coordination task, when enthusiasm of trustful smart objects participating improve using Contract-Net

Model (CNM), Dynamic Contract Net Model (DCNM), and scalable coordinated learning model. In case of system efficiency, enthusiasm of trustful nodes participating is calculated according to the correctly send and received service information, when cooperative team exist in baleful nodes. Results show that scalable coordinated learning model is more capable of improving enthusiasm of trustful nodes participating than CNM and DCNM. Those results further verify significance of optimize dynamic coordination task by decision-making policy with scalable coordinated learning from another paradigms. In addition, there are several factors which affect to the global efficiency of the system in a WSN. On the one hand, the more are the strategy and attribute, the best cooperative team is dynamically formed in order to execute task. On the other hand, the meta-level interaction becomes unstable and unreliable in certain situations (e.g., reflective objects in the scenario which weaken the radio signal or excess traffic which produces an overflow in TX/Rx buffers). This factor is especially critical in meta-level communication among objects, because it can become a bottleneck.

4.3 Enhancing availability by scalable coordinated learning

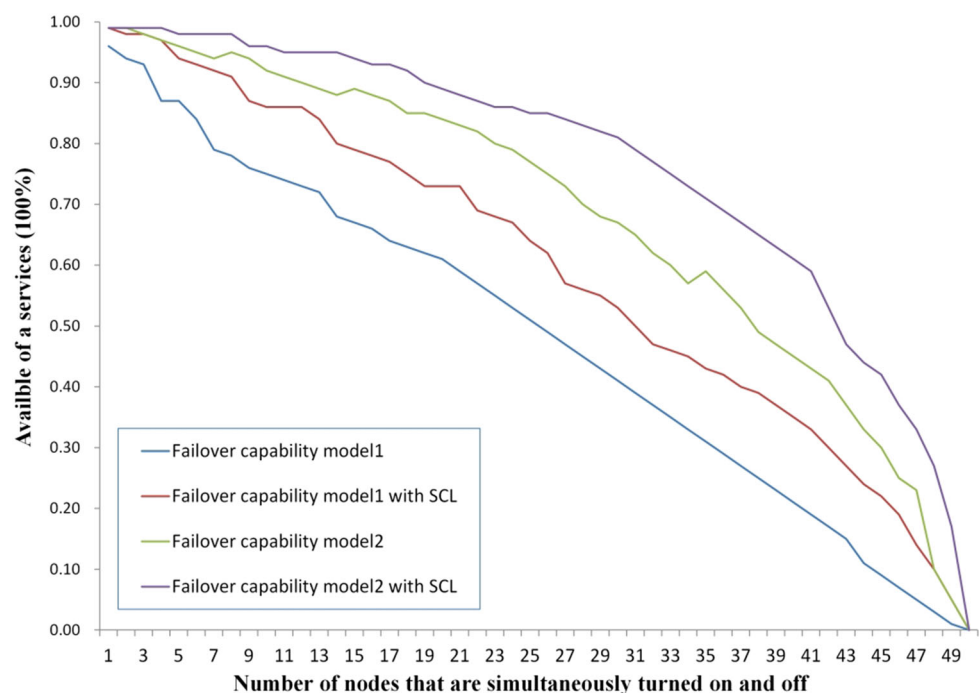
The purpose of this experiment was to test service robustness in the smart agriculture environment where nodes of smart objects could be randomly turned off and on. For finishing the text, seven different services were used and each service was managed by a coordination team. The environment consisted of 60 objects that were

evenly distributed smart agriculture environment. Each node is given different availability, cost, and performance values. The service availability is benchmarked against the availability of services that used a system without fail-over capability and a system with fail-over capability with 2 and 3 redundant objects in the WSNs cluster environment.

Figure 6 shows more capable of enhancing service robustness than the system without fail-over capability (model 1) and the system with two machines to enable fail-over capability (model 2). Although the scalable coordinated learning did not increase service robustness to the level of having a fail-over capability with three machines (model 3), the main difference between scalable coordinated learning and conventional fail-over systems is that scalable coordinated learning does not require the administrator to specify the placement of all the services in advance.

At the same time, Fig. 5 shows the results from the experiment. The line model 1 shows the probability that the service is available without fail-over capability. Model 2 and model 3 show the probability that the service is available with fail-over capabilities using 2 and 3 objects. From the results, it shows that scalable coordinated learning has the probabilities of 77.2 and 76.4 %, respectively, that their services were still available when 52 out of 60 objects (approximately 87 %) were removed from the resource pool. Services running on 3 redundant object (model 3) only had the probability of 36.4 % that its services are still robustness. To conclude, this experiment showed that DSPR was capable of teaming up resources to improve the performance of a service. However, scalable

Fig. 6 The paradigms availability of scalable coordinated learning



coordinated learning was not able to mask the errors caused by the removal of objects completely if the performance required by the service could not be satisfied by the remaining team members. Besides that, the scalable coordinated learning framework needs some time to detect the failure or removal of a server.

5 Conclusion

In this paper, a new approach of multi-objects meta-level coordinated learning is proposed to take advantages of the simplicity and scalability of cooperative multi-objects in IoT. Based on the maximum potential loss for lack of coordination, improving coordinating multi-objects reinforcement learning is made to adapt their behaviors to the dynamics of uncertain and evolving environment in limited communication environment. More importantly, the multi-objects scalable coordinated learning algorithm defines an interaction measure that allows objects to dynamically estimate the potential utility loss for lack of coordination with any cluster of objects. The interaction mechanism make each object compute their beneficial coordination set in different situations and make the best use of their limited communication resource to cooperative multi-objects meta-level learning process. The minimization on objects coordination sets will dramatically reduce links in the object coordination network and often clusters it into smaller connected subnetworks, which allows the algorithms to efficiently compute joint coordination action. The result of experiment shows our multi-objects scalable coordinated learning can scale MARL to co-adapts policy learning of object and their coordination network for different IoT context.

Acknowledgments The author also would like to thank anonymous editor and reviewers who gave valuable suggestion that has helped to improve the quality of the manuscript. This research has been supported by the Project for 2015 National Key Technologies RD Program No. 2015BAH04F 01.

References

- Atzori L, Lera A, Morabito G (2010) The internet of things: a survey. *Comput Netw* 54(15):2787–2805
- Miorandi D, Sicari S, De Pellegrini F, Chlamtac I (2012) The internet of things: vision, applications and research challenges. *Ad Hoc Netw* 10(7):1497–1516
- Cheng SJ, Raja A, Lesser V (2013) Multiagent meta-level control for radar coordination. *Int J Web Intell Agent Syst* 11(1):81–105
- Carlin A, Zilberstein S (2011) Decentralized monitoring of distributed anytime algorithms. In: *Proceedings of the tenth international conference on autonomous agents and multiagent systems (AAMAS 2011)*, Taiwan, pp 157–164
- Zhang C, Lesser VR (2011) Coordinated multi-agent reinforcement learning in networked distributed pomdps. In: *Proceedings of the twenty-fifth national conference on artificial intelligence (AAAI)*, California, USA
- Banerjee B, Peng J (2007) Generalized multiagent learning with performance bound. *Auton Agent Multi-Agent Syst* 15(3):281–312
- Cheng S, Raja A, Lesser V (2010) Multiagent meta-level control for a network of weather radars. In: *Proceedings of IEEE/WIC/ACM international conference on intelligent agent technology (IAT-2010)*, Toronto, Canada, pp 157–164
- Kim Y, Krainin M, Lesse VR (2011) Effective variants of the maxsum algorithm for radar coordination and scheduling. In: *Proceedings of 2011 IEEE/WIC/ACM international conference on intelligent agent technology (IAT-2011)*, Lyon, France, pp 357–364
- Cheng S (2012) Coordinating decentralized learning and conflict resolution across agent boundaries. In PhD thesis, University of North Carolina at Charlotte
- Vinyals M, Rodriguez-Aguilar JA, Cerquides J (2011) A survey on sensor networks from a multiagent perspective. *Comput J* 54(3):455–470
- Raja A, Lesser V (2007) A Framework for meta-level control in multi-agent systems. *Auton Agents and Multi-Agent Syst* 15(2):147–196
- Kennedy C (2010) Decentralised metacognition in context-aware autonomic systems: some key challenges. In: *Proceedings of workshops at the twenty-fourth AAAI conference on artificial intelligence*
- Todd H, Peter S (2009) Generalized model learning for reinforcement learning in factored domains. In: *Proceedings of international conference on autonomous agents and multiagent systems*, pp 490–496
- Zhang C, Lesser V, Abdallah S (2010) Self-organization for coordinating decentralized reinforcement learning. In *AAMAS10*
- Zhang CJ, Lesser V (2013) Coordinating multi-agent reinforcement learning with limited communication. In: *Proceedings of the 12th international conference on autonomous agents and multi-agent systems*, pp 1101–1108
- Conitzer V, Sandholm T (2007) Awesome: a general multiagent learning algorithm that converges in self-play and learns a best response against stationary opponents. *Mach Learn* 67(2):23–43
- Azizi E (2013) Joint learning of modular structures from multiple data types. In *NIPS workshop of frontiers of network analysis: methods, models, and applications*
- Soufiani HA, Airolidi EM (2012) Graphlet decomposition of a weighted network. *J Mach Learn Res JMLR WCP* 22:5463
- Chiang M, Low SH, Doyle JC (2007) Layering as optimization decomposition: a mathematical theory of network architectures. *Proc IEEE* 95(1):255–312
- Wang J, Li L, Low SH, Doyle JC (2005) Cross-layer optimization in TCP/IP networks. *Trans Netw IEEE/ACM* 13(3):582–595
- Yen N, Tung T, Tomi S, Yun T (2012) Transferring expectations in model-based reinforcement learning. In: *Proceedings of the advances in neural information processing systems, NIPS12*
- OSGi HomePage <http://www.osgi.org/>
- Aiello F, Fortino G, Gravina R, Guerrieri A (2011) A Java-based agent platform for programming wireless sensor networks. *Comput J* 54(3):439–454
- Busoniu L, Babuska R, Schutte BD (2008) A comprehensive survey of multiagent reinforcement learning. *IEEE Trans Syst Man Cybern Part C* 38(2):156–172
- Stranders R, Farinelli A, Rogers A, Jennings NR (2009) Decentralised coordination of mobile sensors using the max-sum algorithm. In *IJCAI*, pp 299–304
- Varakantham P, Tambe M, Yokoo M (2005) Networked distributed pomdps: A synthesis of distributed constraint optimization and pomdps. In *AAAI*, pp 133–139